



Guía Definitiva de Extremo a Extremo: "Entrega Final Full-Stack" (10/10)

Esta es la **hoja de ruta maestra e integral** para consolidar los repositorios de tu Proyecto Integrador, tomando como arquitectura de referencia directa el ecosistema oficial del curso **NeonPulse**:

- **Core de Dominio en Java Puro (TDD / Hito 1):** [neonpulse-ticketera](#)
- **Microservicio Backend Spring Boot + Docker (Hito 3 / Hito 4):** [neonpulse-api-springboot](#)
- **Frontend Interactivo Vite + TypeScript (Hito 2):** [neonpulse-frontend](#)

PARTE 1: ARQUITECTURA Y CONTRATO UNIFICADO

Para evitar inconsistencias entre cliente y servidor, establecemos un contrato estandarizado para toda la solución:

Modelo de Dominio Compartido (Referencia: NeonPulse - Gestión de Conciertos y Tickets)

- **Entidad Central:** Concert / Booking (ID, title/band, customerName, email, seats/tickets, status, ticketPrice, createdAt).
- **Estados (ConcertStatus / BookingStatus):** SCHEDULED, LIVE, FINISHED o PENDING, CONFIRMED, CANCELLED.
- **Puertos de Red:**
 - **Frontend (Vite):** <http://localhost:5173>
 - **Backend (Spring Boot):** <http://localhost:8080>
 - **Base de Datos (PostgreSQL en Docker):** <localhost:5432>
 - **Swagger-UI (Solo Dev):** <http://localhost:8080/swagger-ui.html>

PARTE 2: REPOSITARIOS BACKEND Y CORE JAVA

```
backend/
├── src/
│   ├── main/
│   │   ├── java/com/neonpulse/
│   │   │   ├── domain/ # 1. CAPA DE DOMINIO (Java Puro - Cero frameworks)
│   │   │   │   ├── entity/Booking.java / Concert.java
│   │   │   │   └── valueobject/Email.java, Seats.java
```



🔗 Repositorios de referencia de código:

- Revisa la lógica aislada de dominio en Java puro y TDD en:
github.com/sebavidal10/neonpulse-ticketera
- Revisa la arquitectura por capas, JPA, Docker Compose y OpenAPI en:
github.com/sebavidal10/neonpulse-api-springboot

Paso B.1: Dependencias en pom.xml (Maven)

XML

```
<dependencies>
  <!-- Spring Web & Validation -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
```

```

</dependency>

<!-- Persistencia Relacional -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- Documentación OpenAPI / Swagger -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.6.0</version>
</dependency>

<!-- Testing Suite (JUnit 5 + Mockito) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
</dependencies>

```

Paso B.2: Capa de Dominio Pura (DDD Táctico sin Anotaciones de Spring)

```

Java
// domain/valueobject/Email.java
package com.neonpulse.domain.valueobject;

import com.neonpulse.domain.exception.InvalidBookingException;

public record Email(String value) {
    public Email {
        if (value == null || value.trim().isEmpty() || !value.contains("@")) {
            throw new InvalidBookingException("The provided email format is invalid.");
        }
        value = value.trim().toLowerCase();
    }
}

```

Java

// domain/entity/Booking.java

package com.neonpulse.domain.entity;

import com.neonpulse.domain.valueobject.Email;

import com.neonpulse.domain.exception.InvalidBookingException;

import java.time.LocalDateTime;

public class Booking {

private final String id;

private final String customerName;

private final Email email;

private final int seats;

private String status;

private final LocalDateTime createdAt;

public Booking(String id, String customerName, Email email, int seats, String status, LocalDateTime
 createdAt) {

if (customerName == null || customerName.trim().isEmpty()) {

throw new InvalidBookingException("Customer name cannot be empty.");

}

if (seats <= 0 || seats > 10) {

throw new InvalidBookingException("Seats must be between 1 and 10.");

}

this.id = id;

this.customerName = customerName;

this.email = email;

this.seats = seats;

this.status = (status != null) ? status : "CONFIRMED";

this.createdAt = (createdAt != null) ? createdAt : LocalDateTime.now();

}

public String getId() { return id; }

public String getCustomerName() { return customerName; }

public Email getEmail() { return email; }

public int getSeats() { return seats; }

public String getStatus() { return status; }

public LocalDateTime getCreatedAt() { return createdAt; }

}

Java

```
// domain/repository/BookingRepository.java
package com.neonpulse.domain.repository;
```

```
import com.neonpulse.domain.entity.Booking;
import java.util.List;
import java.util.Optional;
```

```
public interface BookingRepository {
    Booking save(Booking booking);
    List<Booking> findAll();
    Optional<Booking> findById(String id);
}
```

Paso B.3: Capa de Aplicación (Casos de Uso)

Java

```
// application/usecase/CreateBookingUseCase.java
package com.neonpulse.application.usecase;
```

```
import com.neonpulse.domain.entity.Booking;
import com.neonpulse.domain.repository.BookingRepository;
import com.neonpulse.domain.valueobject.Email;
import java.util.UUID;
```

```
public class CreateBookingUseCase {
    private final BookingRepository repository;

    public CreateBookingUseCase(BookingRepository repository) {
        this.repository = repository;
    }

    public Booking execute(String customerName, Email email, int seats) {
        String generatedId = UUID.randomUUID().toString();
        Booking newBooking = new Booking(generatedId, customerName, email, seats, "CONFIRMED",
null);
        return repository.save(newBooking);
    }
}
```

Paso B.4: Capa de Infraestructura (Persistencia JPA + PostgreSQL)

Java

```
// infrastructure/persistence/entity/BookingJpaEntity.java
```

```
package com.neonpulse.infrastructure.persistence.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "bookings")
public class BookingJpaEntity {
    @Id
    private String id;

    @Column(nullable = false)
    private String customerName;

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private int seats;

    @Column(nullable = false)
    private String status;

    @Column(nullable = false)
    private LocalDateTime createdAt;

    public BookingJpaEntity() {}

    public BookingJpaEntity(String id, String customerName, String email, int seats, String status,
LocalDateTime createdAt) {
        this.id = id;
        this.customerName = customerName;
        this.email = email;
        this.seats = seats;
        this.status = status;
        this.createdAt = createdAt;
    }

    public String getId() { return id; }
    public String getCustomerName() { return customerName; }
    public String getEmail() { return email; }
    public int getSeats() { return seats; }
    public String getStatus() { return status; }
    public LocalDateTime getCreatedAt() { return createdAt; }
}
```

Java

```
// infrastructure/persistence/adapters/BookingRepositoryAdapter.java
package com.neonpulse.infrastructure.persistence.adapters;
```

```
import com.neonpulse.domain.entity.Booking;
import com.neonpulse.domain.repository.BookingRepository;
import com.neonpulse.domain.valueobject.Email;
import com.neonpulse.infrastructure.persistence.entity.BookingJpaEntity;
import com.neonpulse.infrastructure.persistence.repository.BookingJpaRepository;
import org.springframework.stereotype.Component;
```

```
import java.util.List;
import java.util.Optional;
```

@Component

```
public class BookingRepositoryAdapter implements BookingRepository {
```

```
    private final BookingJpaRepository jpaRepository;
```

```
    public BookingRepositoryAdapter(BookingJpaRepository jpaRepository) {
        this.jpaRepository = jpaRepository;
    }
```

@Override

```
    public Booking save(Booking booking) {
        BookingJpaEntity entity = new BookingJpaEntity(
            booking.getId(),
            booking.getCustomerName(),
            booking.getEmail().value(),
            booking.getSeats(),
            booking.getStatus(),
            booking.getCreatedAt()
        );
        BookingJpaEntity saved = jpaRepository.save(entity);
        return mapToDomain(saved);
    }
```

@Override

```
    public List<Booking> findAll() {
        return jpaRepository.findAll().stream().map(this::mapToDomain).toList();
    }
```

@Override

```
    public Optional<Booking> findById(String id) {
        return jpaRepository.findById(id).map(this::mapToDomain);
    }
```

```

    }

    private Booking mapToDomain(BookingJpaEntity entity) {
        return new Booking(
            entity.getId(),
            entity.getCustomerName(),
            new Email(entity.getEmail()),
            entity.getSeats(),
            entity.getStatus(),
            entity.getCreatedAt()
        );
    }
}

```

Paso B.5: Controladores REST, CORS y Manejo Global de Errores

Java

```

// infrastructure/web/dto/CreateBookingRequest.java
package com.neonpulse.infrastructure.web.dto;

```

```

public record CreateBookingRequest(String customerName, String email, int seats) {}

```

Java

```

// infrastructure/web/dto/ErrorResponse.java
package com.neonpulse.infrastructure.web.dto;

```

```

import java.time.LocalDateTime;

```

```

public record ErrorResponse(String message, String errorCode, LocalDateTime timestamp) {}

```

Java

```

// infrastructure/web/controller/GlobalExceptionHandler.java
package com.neonpulse.infrastructure.web.controller;

```

```

import com.neonpulse.domain.exception.InvalidBookingException;
import com.neonpulse.infrastructure.web.dto.ErrorResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.time.LocalDateTime;

```

```

@RestControllerAdvice
public class GlobalExceptionHandler {

```



```

@ExceptionHandler(InvalidBookingException.class)
public ResponseEntity<ErrorResponse> handleBusinessException(InvalidBookingException ex) {
    ErrorResponse error = new ErrorResponse(
        ex.getMessage(),
        "BUSINESS_RULE_VIOLATION",
        LocalDateTime.now()
    );
    return new ResponseEntity<>(error, HttpStatus.UNPROCESSABLE_ENTITY); // 422
}

```

```

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGenericException(Exception ex) {
    ErrorResponse error = new ErrorResponse(
        "An internal error occurred. Please contact support.",
        "INTERNAL_SERVER_ERROR",
        LocalDateTime.now()
    );
    return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR); // 500
}

```

Sanitizado

```

}
}

```

Java

```

// infrastructure/web/controller/BookingController.java
package com.neonpulse.infrastructure.web.controller;

```

```

import com.neonpulse.application.usecase.CreateBookingUseCase;
import com.neonpulse.application.usecase.ListBookingsUseCase;
import com.neonpulse.domain.entity.Booking;
import com.neonpulse.domain.valueobject.Email;
import com.neonpulse.infrastructure.web.dto.CreateBookingRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```

import java.util.List;

```

```

@RestController
@RequestMapping("/api/v1/bookings")
@CrossOrigin(origins = {"http://localhost:5173", "http://localhost:3000"}) // Cero errores CORS
public class BookingController {

```

```

    private final CreateBookingUseCase createUseCase;
    private final ListBookingsUseCase listUseCase;

```

```

    public BookingController(CreateBookingUseCase createUseCase, ListBookingsUseCase
listUseCase) {
        this.createUseCase = createUseCase;
        this.listUseCase = listUseCase;
    }

    @GetMapping
    public ResponseEntity<List<Booking>> listAll() {
        return ResponseEntity.ok(listUseCase.execute());
    }

    @PostMapping
    public ResponseEntity<Booking> create(@RequestBody CreateBookingRequest request) {
        Booking created = createUseCase.execute(
            request.customerName(),
            new Email(request.email()),
            request.seats()
        );
        return new ResponseEntity<>(created, HttpStatus.CREATED); // 201 Created
    }
}

```

Paso B.6: Configuración de Perfiles y Ciberseguridad (application.yml)

YAML

resources/application.yml (Gestión de Variables de Entorno)

```

spring:
  application:
    name: booking-service
  profiles:
    active: ${SPRING_PROFILES_ACTIVE:dev}
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/booking_db}
    username: ${DB_USER:dev_user}
    password: ${DB_PASSWORD:SecretDev123}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false

```

YAML

resources/application-dev.yml (Perfil de Desarrollo Local con Swagger Activo)

```

springdoc:
  api-docs:

```

```
path: /api-docs
enabled: true
swagger-ui:
  path: /swagger-ui.html
  enabled: true
try-it-out-enabled: true
```

YAML

```
# resources/application-prod.yml (Perfil de Producción Seguro - Swagger Bloqueado)
springdoc:
  api-docs:
    enabled: false
  swagger-ui:
    enabled: false
```

Paso B.7: Docker Compose para Base de Datos

YAML

```
# docker-compose.yml (Raíz del proyecto backend)
version: '3.8'
services:
  db:
    image: postgres:16-alpine
    container_name: booking-postgres-db
    restart: always
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: booking_db
      POSTGRES_USER: dev_user
      POSTGRES_PASSWORD: SecretDev123
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```
volumes:
  postgres_data:
```

Paso B.8: Suite de Pruebas TDD (JUnit 5 + Mockito)

Java

```
// test/java/com/neonpulse/application/CreateBookingUseCaseTest.java
package com.neonpulse.application;
```

```
import com.neonpulse.application.usecase.CreateBookingUseCase;
import com.neonpulse.domain.entity.Booking;
```

```
import com.neonpulse.domain.exception.InvalidBookingException;
import com.neonpulse.domain.repository.BookingRepository;
import com.neonpulse.domain.valueobject.Email;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class CreateBookingUseCaseTest {

    @Mock
    private BookingRepository repository;

    @InjectMocks
    private CreateBookingUseCase useCase;

    @Test
    void shouldCreateBookingSuccessfully() {
        // Arrange
        Email validEmail = new Email("alex@example.com");
        when(repository.save(any(Booking.class))).thenReturn(invocation -> invocation.getArgument(0));

        // Act
        Booking result = useCase.execute("Alex Turner", validEmail, 4);

        // Assert
        assertNotNull(result.getId());
        assertEquals("Alex Turner", result.getCustomerName());
        assertEquals(4, result.getSeats());
        assertEquals("CONFIRMED", result.getStatus());
        verify(repository, times(1)).save(any(Booking.class));
    }

    @Test
    void shouldThrowExceptionWhenCustomerNameIsInvalid() {
        // Arrange
        Email validEmail = new Email("alex@example.com");

        // Act & Assert
```

```

    assertThrows(InvalidBookingException.class, () -> {
        useCase.execute("", validEmail, 4);
    });
    verify(repository, never()).save(any(Booking.class));
}
}

```

PARTE 3: REPOSITORIO FRONTEND (Vite + TypeScript Vanilla)

📌 Repositorio de referencia de código frontend:

- Revisa la estructura modular, interfaces y llamadas Fetch asíncronas en: github.com/sebavidal10/neonpulse-frontend

```

frontend/
├── src/
│   ├── models/           # 1. MODELOS E INTERFACES (Cero 'any', Enums)
│   │   └── booking.ts
│   ├── services/         # 2. CONSUMO ASÍNCRONO FETCH (Try/Catch)
│   │   └── bookingApi.ts
│   ├── components/       # 3. COMPONENTES VISUALES MODULARES
│   │   ├── BookingCard.ts
│   │   └── Alert.ts
│   ├── style.css          # 4. ESTILOS Y FEEDBACK VISUAL
│   └── main.ts            # 5. CONTROLADOR DOM Y EVENTOS
├── index.html
├── .gitignore
├── package.json
├── tsconfig.json
└── README.md

```

Paso F.1: Modelado Estricto de Tipos (src/models/booking.ts)

TypeScript

```

export enum BookingStatus {
    PENDING = "PENDING",
    CONFIRMED = "CONFIRMED",
    CANCELLED = "CANCELLED"
}

```

```

export interface Booking {
    id: string;
}

```

```

customerName: string;
email: { value: string } | string; // Soporta Value Object deserializado
seats: number;
status: BookingStatus;
createdAt: string;
}

```

```

export interface CreateBookingDTO {
  customerName: string;
  email: string;
  seats: number;
}

```

Paso F.2: Servicio de Red Asíncrono (src/services/bookingApi.ts)

TypeScript

```

import { Booking, CreateBookingDTO } from "../models/booking";

```

```

const BASE_URL = "http://localhost:8080/api/v1/bookings";

```

```

export async function fetchBookings(): Promise<Booking[]> {
  const response = await fetch(BASE_URL);
  if (!response.ok) {
    throw new Error(`Failed to load bookings: HTTP ${response.status}`);
  }
  return (await response.json()) as Booking[];
}

```

```

export async function createBooking(dto: CreateBookingDTO): Promise<Booking> {
  const response = await fetch(BASE_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dto)
  });

```

```

  if (!response.ok) {
    const errorBody = await response.json().catch(() => null);
    throw new Error(errorBody?.message || `Server Error: HTTP ${response.status}`);
  }

```

```

  return (await response.json()) as Booking;
}

```

Paso F.3: Componentes Visuales Modulares

(src/components/BookingCard.ts)

TypeScript

```
import { Booking } from "../models/booking";

export function generateBookingCardHtml(booking: Booking): string {
  const emailText = typeof booking.email === "object" ? booking.email.value : booking.email;
  return `
    <article class="booking-card">
      <div class="card-header">
        <h3>${booking.customerName}</h3>
        <span class="badge ${booking.status.toLowerCase()}">${booking.status}</span>
      </div>
      <p><strong>Email:</strong> ${emailText}</p>
      <p><strong>Seats Reserved:</strong> ${booking.seats}</p>
      <small><strong>ID:</strong> ${booking.id}</small>
    </article>
  `;
}
```

Paso F.4: HTML Base (index.html)

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Enterprise Booking System</title>
    <link rel="stylesheet" href="../src/style.css" />
  </head>
  <body>
    <header class="app-header">
      <h1>📅 Booking Management System</h1>
      <p>Integrated Full-Stack Application</p>
    </header>

    <main class="container">
      <!-- Formulario de Entrada -->
      <section class="form-section">
        <h2>Reserve Seats</h2>
        <form id="booking-form" novalidate>
          <div class="form-group">
            <label for="txt-name">Customer Name:</label>
            <input type="text" id="txt-name" placeholder="e.g. Alex Turner" required />
          </div>
        </form>
      </section>
    </main>
  </body>
</html>
```

```

</div>

<div class="form-group">
  <label for="txt-email">Email Address:</label>
  <input type="email" id="txt-email" placeholder="alex@domain.com" required />
</div>

<div class="form-group">
  <label for="txt-seats">Seats (1-10):</label>
  <input type="number" id="txt-seats" min="1" max="10" value="1" required />
</div>

<button type="submit" id="btn-submit">Submit Reservation</button>
</form>

<div id="feedback-message" class="feedback"></div>
</section>

<!-- Lista de Reservas -->
<section class="list-section">
  <h2>Active Reservations</h2>
  <div id="bookings-list" class="cards-grid">
    <p class="loading">Loading current reservations...</p>
  </div>
</section>
</main>

<script type="module" src="./src/main.ts"></script>
</body>
</html>

```

Paso F.5: Controlador Principal del DOM (src/main.ts)

TypeScript

```

import { CreateBookingDTO } from "../models/booking";
import { fetchBookings, createBooking } from "../services/bookingApi";
import { generateBookingCardHtml } from "../components/BookingCard";

const bookingForm = document.getElementById("booking-form") as HTMLFormElement | null;
const nameInput = document.getElementById("txt-name") as HTMLInputElement | null;
const emailInput = document.getElementById("txt-email") as HTMLInputElement | null;
const seatsInput = document.getElementById("txt-seats") as HTMLInputElement | null;
const feedbackContainer = document.getElementById("feedback-message");
const listContainer = document.getElementById("bookings-list");

```



```

// 1. Cargar datos remotos al iniciar
async function loadAndRenderBookings(): Promise<void> {
  if (!listContainer) return;
  listContainer.innerHTML = "<p class='loading'>Fetching reservations from PostgreSQL...</p>";

  try {
    const bookings = await fetchBookings();
    if (bookings.length === 0) {
      listContainer.innerHTML = "<p class='empty-state'>No reservations registered yet.</p>";
      return;
    }
    listContainer.innerHTML = bookings.map(generateBookingCardHtml).join("");
  } catch (error: any) {
    listContainer.innerHTML = `
      <div class="error-box">
        <p><strong>Error loading data:</strong></p>
        <small>${error.message}</small>
      </div>
    `;
  }
}

```

```

// 2. Manejo reactivo de envíos del formulario
if (bookingForm) {
  bookingForm.addEventListener("submit", async (event: Event) => {
    event.preventDefault(); // Detener recarga nativa

    if (!nameInput || !emailInput || !seatsInput || !feedbackContainer) return;

    const customerName = nameInput.value.trim();
    const email = emailInput.value.trim();
    const seats = parseInt(seatsInput.value, 10);

    // Validaciones estrictas en cliente
    if (customerName.length === 0) {
      showFeedback("Customer name cannot be empty.", "error");
      return;
    }
    if (!email.includes("@") || email.length < 5) {
      showFeedback("Please provide a valid email address.", "error");
      return;
    }
    if (isNaN(seats) || seats <= 0 || seats > 10) {
      showFeedback("Seats must be a number between 1 and 10.", "error");
      return;
    }
  });
}

```

```

    }

    const payload: CreateBookingDTO = { customerName, email, seats };

    try {
      showFeedback("Processing reservation...", "info");
      await createBooking(payload);
      showFeedback("Reservation successfully created and stored in PostgreSQL!", "success");

      // Limpiar campos
      bookingForm.reset();
      seatsInput.value = "1";

      // Actualizar lista en tiempo real
      await loadAndRenderBookings();
    } catch (error: any) {
      showFeedback(error.message, "error");
    }
  });
}

function showFeedback(message: string, type: "error" | "success" | "info"): void {
  if (!feedbackContainer) return;
  feedbackContainer.className = `feedback ${type}`;
  feedbackContainer.textContent = message;
}

// Inicialización
document.addEventListener("DOMContentLoaded", loadAndRenderBookings);

```

PARTE 4: CIBERSEGURIDAD, .gitignore Y PRODUCCIÓN

Para cumplir con el **Pilar 3 (Seguridad de Grado de Producción)**:

1. .gitignore Estricto en Backend

Fragmento de código

```

target/
*.class
.env
.env.*
application-prod.yml

```

.idea/
*.iml
.vscode/
.DS_Store

2. .gitignore Estricto en Frontend

Fragmento de código
node_modules/
dist/
dist-ssr/
*.local
.env
.env.*
.DS_Store

PARTE 5: CHECKLIST DE AUTORREVISIÓN ANTES DE SUBIR A GITHUB

#	Validación	Comando / Verificación	Puntos Rúbrica
1	Pruebas Unitarias	Ejecutar ./mvnw clean test -> 100% pasando	3 Pts (DDD+TDD)
2	Cero Frameworks en Dominio	Ningún @Entity, @Table ni anotación Spring en domain/	3 Pts (DDD+TDD)
3	Persistencia Real	docker compose up -d -> BD activa en puerto 5432	4 Pts (Integración)
4	CORS Resuelto	@CrossOrigin(origins = "http://localhost:5173") en Backend	4 Pts (Integración)
5	Cero any en Frontend	Ejecutar npm run build sin errores de compilación TS	4 Pts (Integración)
6	Ciclo Completo	Formulario web inserta en PostgreSQL y refresca la UI	4 Pts (Integración)

7	Exclusión de Secretos	Cero contraseñas de producción o .env subidos a Git	3 Pts (Seguridad)
8	Swagger Aislado	Swagger bloqueado bajo prod y activo bajo dev	3 Pts (Seguridad)

PARTE 6: README.MD PROFESIONAL PARA CADA REPOSITORIO

Tanto el repositorio Backend como el Frontend deben tener un archivo README.md con esta estructura exacta:

Markdown

📖 Enterprise Booking System - Full-Stack Integration

🛠 Stack Tecnológico

* **Backend:** Java 21, Spring Boot 3, Spring Data JPA, Hibernate, OpenAPI/Swagger.
 * **Frontend:** TypeScript Vanilla, Vite, Native ESM, HTML5/CSS3 Semántico.
 * **Infraestructura:** Docker Compose, PostgreSQL 16 Alpine.
 * **Calidad y Testing:** JUnit 5, Mockito, JaCoCo, TDD & Clean Architecture.

🔗 Repositorios de Referencia

* Core de Dominio / Hito 1: <https://github.com/sebavidal10/neonpulse-ticketera>
 * Backend Spring Boot / Hito 4: <https://github.com/sebavidal10/neonpulse-api-springboot>
 * Frontend Vite + TS / Hito 2: <https://github.com/sebavidal10/neonpulse-frontend>

🚀 Guía de Puesta en Marcha Local

1. Levantar la Base de Datos Relacional

```
``bash
cd backend
docker compose up -d
```

2. Ejecutar Pruebas Automatizadas

`./mvnw clean test`

3. Iniciar el Microservicio Backend

`./mvnw spring-boot:run`

- API REST: <http://localhost:8080/api/v1/bookings>
- Swagger UI (Perfil Dev): <http://localhost:8080/swagger-ui.html>

4. Iniciar la Interfaz Web Frontend

`cd ../frontend`

`npm install`

`npm run dev`

- App Web: <http://localhost:5173>